

Bit flips per operation

Chaya2766

December 6, 2025

Abstract

Abstract goes here (todo)

Contents

1	Assumptions	2
2	Addition and Subtraction	3
3	Multiplication (integer and fixed-point)	4
4	Division (todo)	6
5	Final gate count	7

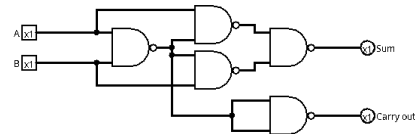
1 Assumptions

Why NAND gates?

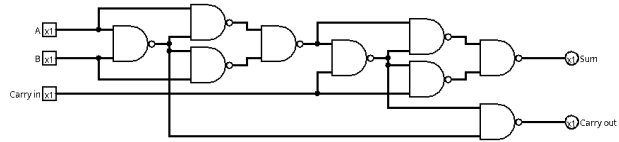
1. My whole system of FPGA-based "computronium" relies on a paper that among other things estimated the energy to flip a nand gate made using a number of different computing forms (eg. standard CMOS, other electronic, spintronic, nanomagnet logic, etc.).
2. NAND gates alone can be used to construct any circuit. When combined the right way they can approximate any other gate.

2 Addition and Substraction

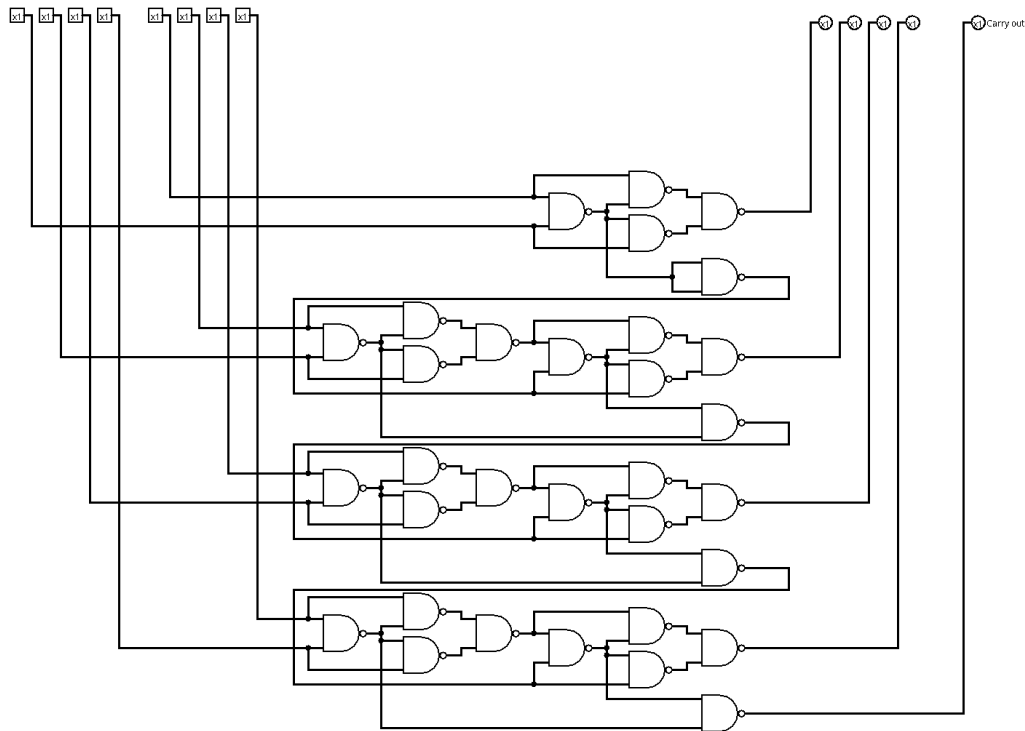
The half adder is used for the first layer of addition (ie. the first bit of both numbers) and has only 5 NAND gates.



The full adder consists of 9 NAND gates and has to be used for each next bit after the first.



Here is an example 4-bit adder with 32 NAND gates:



This shows that the number of NAND gates in an adder circuit can be described as such:

$$G = 5 + (n - 1) \cdot 9 = 9n - 4$$

where:

- G is the number of NAND gates in the adder
- n is the number of bits of the adder

One gate could also be removed in the final step if you don't need the carry bit.

This circuit can also perform subtraction by using values encoded in two's complement.

3 Multiplication (integer and fixed-point)

(big thanks to this video <https://www.youtube.com/embed/O34KquoMpT0>)

The multiplier type I'll be using is simple cascading multiplier that works something like this:

number A				A_2	A_1	A_0
number B				B_2	B_1	B_0
partial 1				A_2B_0	A_1B_0	A_0B_0
partial 2			A_2B_1	A_1B_1	A_0B_1	
partial 3		A_2B_2	A_1B_2	A_0B_2		
sum	P_5	P_4	P_3	P_2	P_1	P_0

A_n represents the nth bit of number A, and the same for number B.

P_n represents the nth bit of the result.

Multiplying two bits (eg. A_0 with B_0 which would be written as just A_0B_0) is an AND operation. Every combination of two bits requires an AND gate, which is made from 2 NAND gates. This part adds $2n^2$ NAND gates to the multiplying circuit.

The additions have to be done with adders such as shown in section 2.

In an n-bit multiplier each row will have length n and there will be n rows

- First row (partial 1): does not require adders.
(0)
- Second row (partial 2): requires a half-adder at the start, a half-adder at the end, and full adders through the middle.
(2 half-adders and n-2 full adders, which is $2 \cdot 5 + (n - 2) \cdot 9 = 9n - 8$ NAND gates)
- Third row (partial 3): requires a half-adder at the start, and full adders the rest of the way.
(1 half adder and n-1 full adders, which is $1 \cdot 5 + (n - 1) \cdot 9 = 9n - 4$ NAND gates)
- Every row after the third is the same as the third, so the third row is additionally multiplied by n-2

In total we need to sum up these terms:

$2n^2$ from the ANDs across the whole thing.

$9n - 8$ from the adders in the second row.

$(n - 2) \cdot (9n - 4) = (9n^2 - 22n + 8)$ from the adders from row 3 onwards.

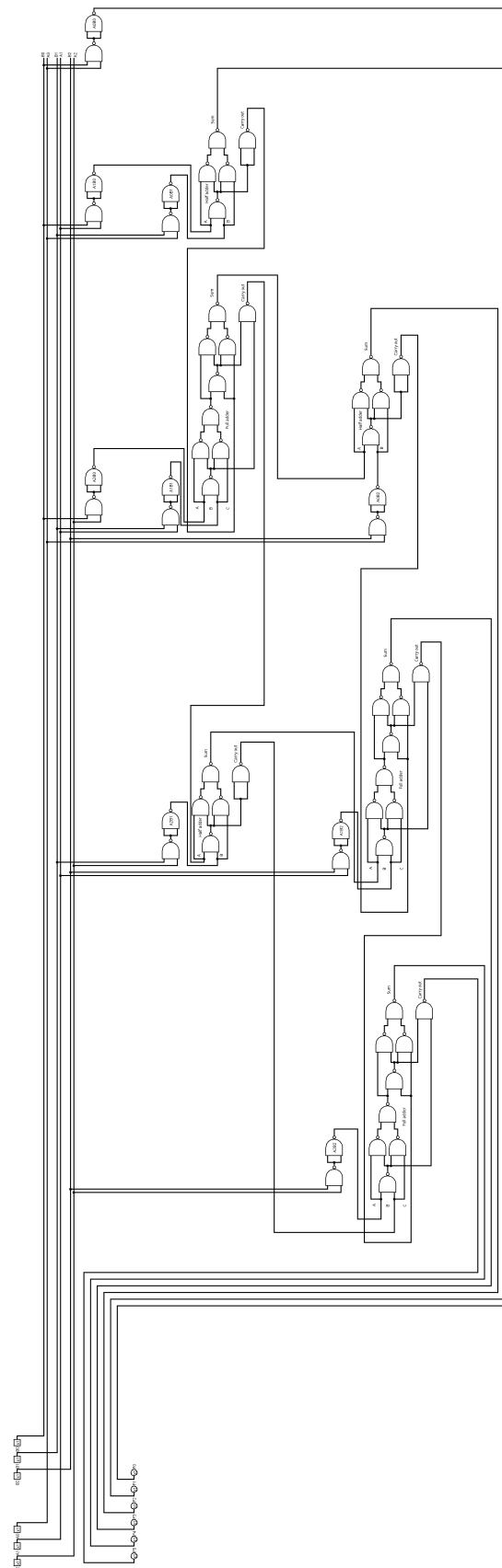
In total an n-bit multiplier should have this many gates:

$$G = 11n^2 - 13n$$

where:

- G is the number of NAND gates in the multiplier
- n is the number of bits of the adder

Here is an example 3-bit multiplier with 60 gates (yes I counted manually to confirm):



4 Division (todo)

5 Final gate count

I've also included gate count for a "multiply and accumulate" circuit, which is simply a multiplication combined with an addition, and I assume the needed circuit has as many gates as those two circuits added together.

$$G_{+/-} = 9n - 4$$

$$G_{\times} = 11n^2 - 13n$$

$$G_{\div} = ???$$

$$G_{+\times} = G_{+/-} + G_{\times} = 11n^2 - 4n - 4$$

where:

- $G_{+/-}$ is the number of NAND gates in an n-bit adder
- $G_{\times}$ is the number of NAND gates in an n-bit multiplier
- $G_{\div}$ is the number of NAND gates in an n-bit divider
- $G_{+\times}$ is the number of NAND gates in an n-bit multiply-and-accumulate circuit
- n is the number of bits in an input variable

There exist optimizations that allow you to get gate counts smaller than that given by these equations, but they haven't been accounted for. These equations give you what you'll get if you just systematically apply the usual method of building these circuits with no additional tweaking.

I included a table with the number of NAND gates in each of those circuits from 1 bits to 32 bits. The row for 1 bit was only included for completeness, as contemplating 1-bit implementation of these circuits largely makes no sense. These should be a good approximation for how many bits you need to flip to perform the given operation.

n	$G_{+/-}$	$G_{\times}$	$G_{\div}$	$G_{+\times}$	n	$G_{+/-}$	$G_{\times}$	$G_{\div}$	$G_{+\times}$
1	5	-2	-	3	17	149	2958	-	3107
2	14	18	-	32	18	158	3330	-	3488
3	23	60	-	83	19	167	3724	-	3891
4	32	124	-	156	20	176	4140	-	4316
5	41	210	-	251	21	185	4578	-	4763
6	50	318	-	368	22	194	5038	-	5232
7	59	448	-	507	23	203	5520	-	5723
8	68	600	-	668	24	212	6024	-	6236
9	77	774	-	851	25	221	6550	-	6771
10	86	970	-	1056	26	230	7098	-	7328
11	95	1188	-	1283	27	239	7668	-	7907
12	104	1428	-	1532	28	248	8260	-	8508
13	113	1690	-	1803	29	257	8874	-	9131
14	122	1974	-	2096	30	266	9510	-	9776
15	131	2280	-	2411	31	275	10 168	-	10 443
16	140	2608	-	2748	32	284	10 848	-	11 132